多相型言語の変数名補完を行う Emacs モードの開発

後藤 拓実　篠埜 功

芝浦工業大学 情報工学科

PPL2010
2010 年 3 月 3〜5 日

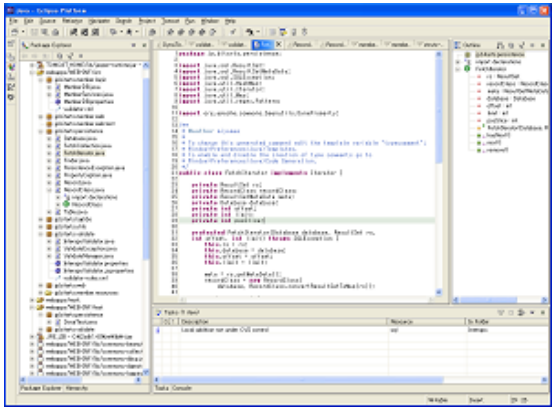
背景

Standard ML、OCaml、Haskell 等の関数型言語は開発環境が整っていない

- ライブラリが少ない
- 統合開発環境が整っていない

統合開発環境の機能：

- Syntax Highlight
- 変数名/メンバー名補完
- 構文エラーのチェック



統合開発環境の例：

- Eclipse (Java, C++, ML-Dev(Standard ML) 等)
- Visual Studio (C++,C#,Basic 等)
- Emacs (JDEE(Java) 等)

研究の目的

関数型言語の開発環境で変数名補完の備わっているものは見当たらない。

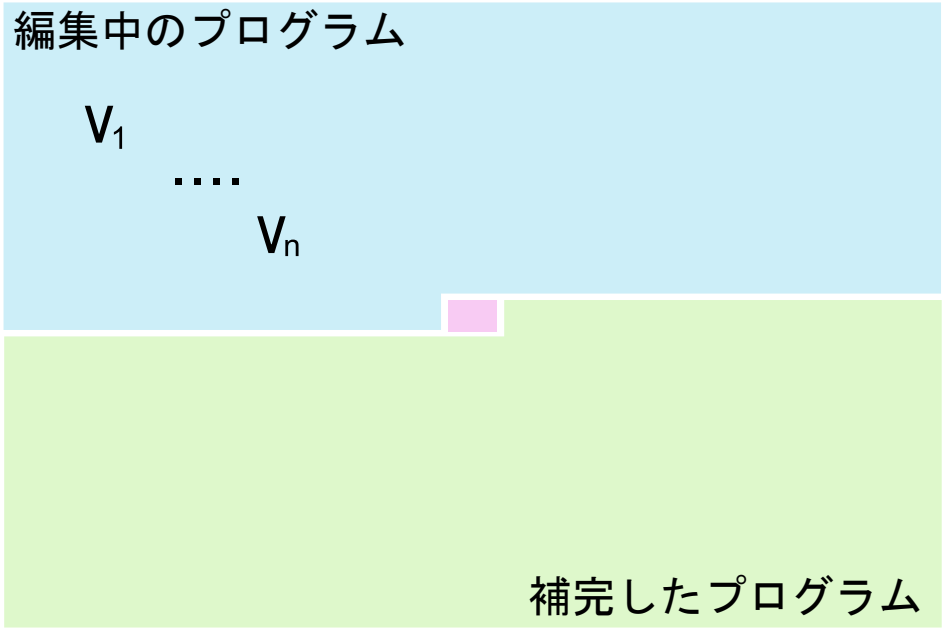
本研究では、型推論機構を備えた言語について変数名補完を行う Emacs モードを開発するための基本的枠組みを提案する。

Eclipse による補完の例 (Java)

```
void F(){
    int number = 2;
    String name = "abc";
    String x = n
```

name String
number int
notify() void - Object

本研究で行う変数名補完について



有効な変数:$\{v_1 \ldots v_n\}$

絞り込み

(型検査, 入力中の字句との比較)

補完候補となる変数

提案手法について

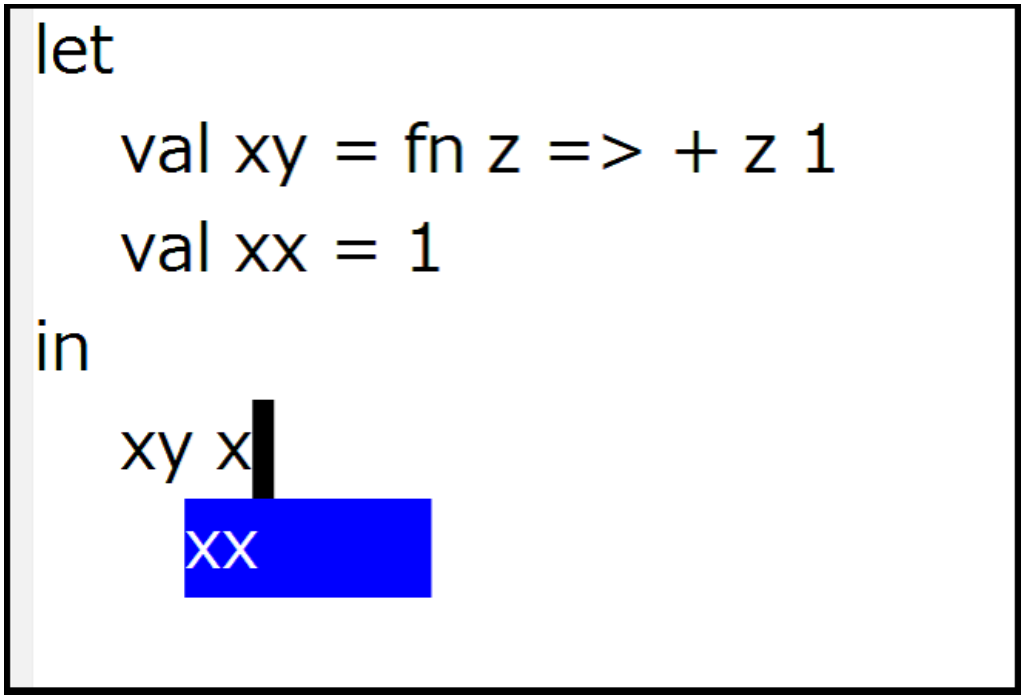
型情報を変数名補完に利用するメリット:

- 入力の手間が減る
- コンパイル時の型エラーを減らせる

型推論機構のある関数型言語では型情報は推論によって得られるので書かないこともある。

そのため型情報を得るために型推論を行う必要がある。

提案手法を実装したシステムのスクリーンショット



対象とする言語の構文定義

$$
\begin{array}{lcl}
\mathit{start} & ::= & \mathit{exp} \\
\mathit{exp} & ::= & \mathit{appexp} \\
 & | & \texttt{fn}\ \mathit{id}\ \Rightarrow \mathit{exp} \\
\mathit{appexp} & ::= & \mathit{atexp} \\
 & | & \mathit{appexp}\ \mathit{atexp} \\
\mathit{atexp} & ::= & \mathit{id} \\
 & | & \mathit{const} \\
 & | & (\mathit{exp}) \\
\mathit{atexp} & ::= & \texttt{let}\ \mathit{dec}\ \texttt{in}\ \mathit{exp}\ \texttt{end} \\
\mathit{dec} & ::= & \texttt{val}\ \mathit{id} = \mathit{exp} \\
 & | & \mathit{dec}\ \mathit{dec}
\end{array}
$$

型システムは通常の ML 系の let 多相の型システムとする。

提案手法の仮定

提案手法ではカーソルより前に記述されているプログラムに構文エラーや型エラーがないと仮定する。
（そのようなエラーがある場合には変数名補完を中断する。）

例 (_はカーソル位置を表す)

- 構文エラー

```
let let val x = 1 _
```

- 型エラー

```
let val x = 1 in x 1 _
```

補完計算の流れ

1. 字句解析
   カーソル位置まで解析しカーソル要素と EOF を加える。
2. 構文解析
   EOF まで解析し、スタック情報と部分的な構文木を得る
3. 構文木補完
   ダミー要素を補い考えられるすべての構文木を作成
4. 型推論
   3 で得られたすべての構文木の変数とカーソル要素の型を推論
5. 候補の絞り込み
   4 で得られた型情報及び入力中の文字列により絞り込む

字句解析

- カーソルを入力中のトークンの直前へと移動。
- プログラムの先頭からカーソル位置までを字句解析。
- 最後に EOF を加える。
- カーソル位置を表すカーソル要素という字句を EOF の手前に加える。カーソル要素の扱いは変数と同じ扱いとする。

字句解析の例

入力プログラムテキスト (_はカーソル位置)

```
let val x =  y_
```

カーソルが移動される。

```
let val x =  _y
```

カーソル位置まで解析し、カーソル要素と EOF を加える。

$\texttt{let}, \texttt{val}, id_{\texttt{x}}, =, id_{\text{cursor}}, \text{EOF}$

構文解析

- LR 構文解析
- EOF を先読みしたら構文解析は終了する。
- 構文解析の結果は終了時のスタックの状態と還元の履歴 (部分的な構文木)。

構文解析の例

入力プログラム

```
let val xy = fn z ⇒ + z 1
    val xx = 1
in xy _
```

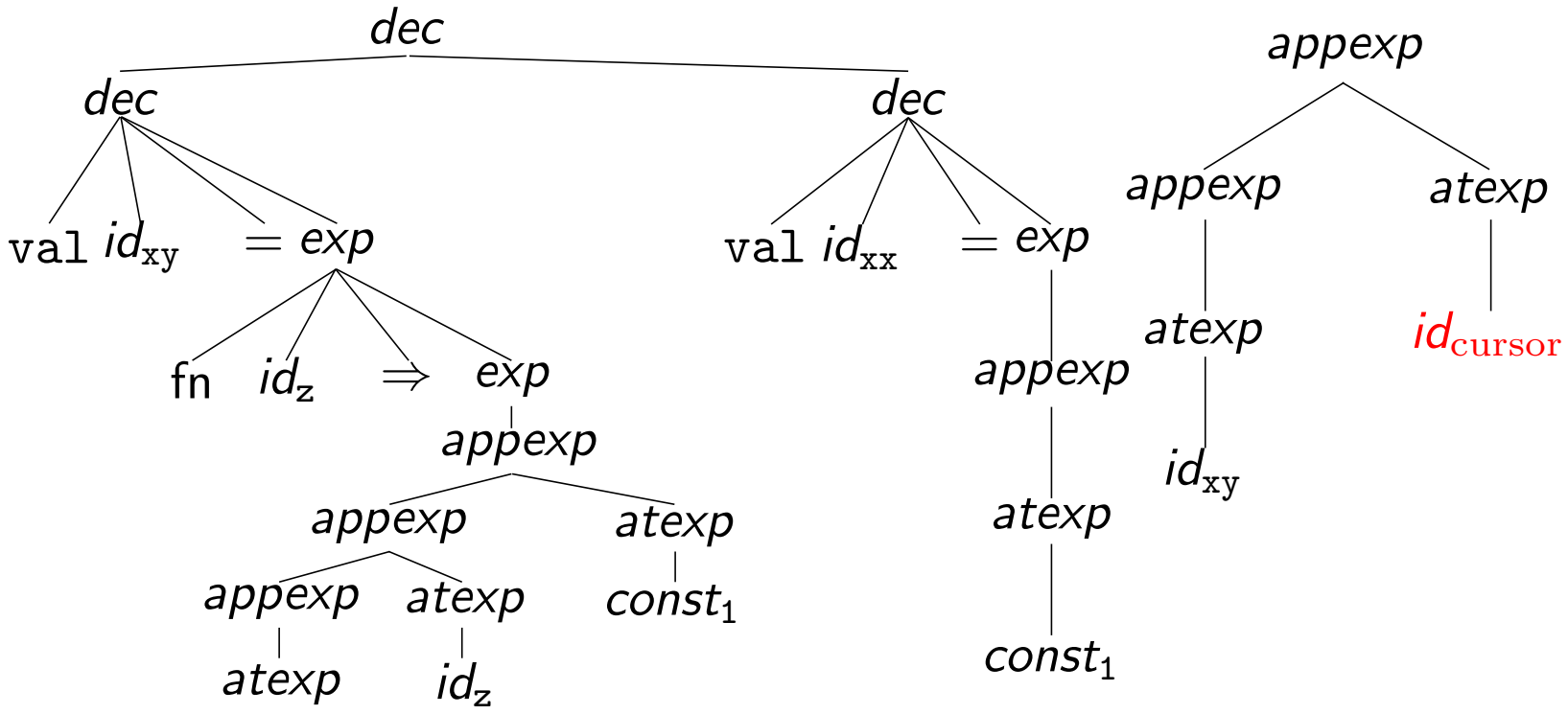
字句解析の結果

$\texttt{let},\ \texttt{val},\ id_{\texttt{xy}},\ =,\ \texttt{fn},\ id_{\texttt{z}},\ \Rightarrow,\ id_{+},\ id_{\texttt{z}},\ const_1$
$\texttt{val},\ id_{\texttt{xx}},\ =,\ const_1,\ \texttt{in},\ id_{\texttt{xy}},\ id_{\text{cursor}},\ \text{EOF}$

構文解析の例

スタックの状態: `let`, *dec*, `in`, *appexp*

部分的な構文木:



構文解析の例

プログラムテキストにすると、(␣はカーソル要素を表す)

```
let val xy = fn z ⇒ + z 1
    val xx = 1
in xy ␣
```

構文木補完

- スタック情報をもとに部分的な構文木にダミーの非終端記号/終端記号を補完し構文木を完成させる。
- 考えられる全ての構文木を作成する。

一般に無限個の構文木が考えられるが、本研究では閾値を用いて有限個の構文木を作成することとした。

構文木補完の例

構文解析の結果

```
let val xy = fn z ⇒ + z 1
    val xx = 1
in xy ␣
```

補完結果の構文木をテキストで表したもの

```
let val xy = fn z ⇒ + z 1
    val xx = 1
in xy ␣ end
```

構文木補完の例

関数適用が書ける部分では何度でもダミー要素が挿入される

```
let val xy = fn z ⇒ + z 1
    val xx = 1
in xy ␣  [] end
```

```
let val xy = fn z ⇒ + z 1
    val xx = 1
in xy ␣ end []
```

型推論

- 構文木補完により作成された複数の構文木に対してそれぞれ型推論を行う
- 推論アルゴリズムは Milner の $\mathcal{W}$ にカーソル要素とダミー要素の場合を追加したもの
- 型推論の過程ではカーソル要素とダミー要素の型は新しい型変数とする。
- 結果としてカーソル要素の型とカーソル位置で有効な変数を得る。

型推論の例

構文木補完後のプログラム

```
let val xy = fn z ⇒ + z 1
    val xx = 1
in xy ␣ end
```

型推論後のプログラム

$$\texttt{let val } \underline{\texttt{xy}}_{int \to int} \texttt{ = fn z} \Rightarrow \texttt{+ z 1}$$
$$\texttt{val } \underline{\texttt{xx}}_{int} \texttt{ = 1}$$
$$\texttt{in } \underline{\texttt{xy}}_{int \to int} \ \texttt{␣}_{int} \ \texttt{end}$$

候補の絞込み

カーソル位置で有効な変数のうち次の条件両方を満たすものを候補として残す

- 変数の型とカーソル位置の型が単一化できる
- プログラマが入力中の文字列と変数名の先頭が一致する

絞込みの例

```
let val xy_{int→int} = fn z ⇒ + z 1
    val xx_{int} = 1
in xy ␣_{int} end
```

- カーソル位置で有効な変数：xy($int \to int$), xx(int)
- カーソル位置の型：int

カーソル位置と有効な変数の型について単一化を行うと

- xy($int \to int$) と ␣(int) は単一化不可
- xx(int) と ␣(int) は単一化可

よって候補として残るのは二つのうち xx のみである。

実装について

- Emacs に Emacs Lisp を用いて実装
- 補完のインタフェースは auto-complete.el

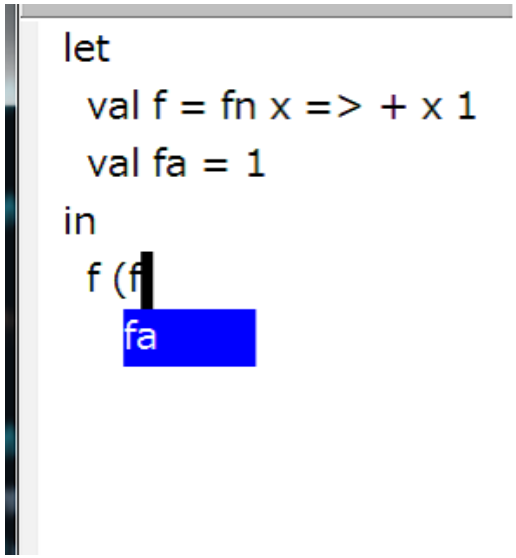
提案手法ではプログラマが 1 文字を入力するごとに候補の計算を行う。

候補が出ない場合

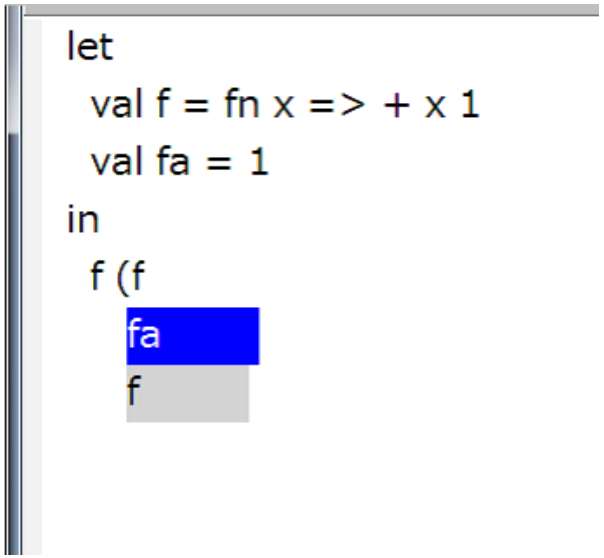
構文木補完で閾値を用いているため出るべき候補がでないことがある

例：

閾値:9



閾値:10



まとめ

関数型言語の核となる部分についての変数名補完を提案し、実装した。

今後の課題

- キーワードの補完も行えるように拡張する。
- カーソル位置より後方で宣言される変数についての補完
  - 相互再帰関数など
- 計算時間の短縮
  - 計算の再利用
  - 構文木補完方式の改良

多相型の単一化の例

$$\begin{array}{l}\texttt{let val } \underline{\texttt{xy}}_{\forall\alpha.\alpha\to\alpha} = \texttt{fn z} \Rightarrow \texttt{z} \\ \texttt{in } \underline{\texttt{xy}}_{\beta\to\beta} \ \underline{=}_{\beta} \ \texttt{end}\end{array}$$

カーソル要素の型：$\underline{=}_{\beta}$
カーソル位置で有効な変数：$\underline{\texttt{xy}}_{\forall\alpha.\alpha\to\alpha}$

変数 xy は多層型であるので具体化を行う。
具体化後の xy の型：$\gamma \to \gamma$

カーソル要素の型 β と変数 xy の型 $\gamma \to \gamma$ は単一化可能である

絞り込みの方法

1、カーソル要素の型を新しい型変数として型推論を行う。2、カーソル要素の型と有効な変数の型に単一化を行い、単一化できるものを候補として残す。

構文解析

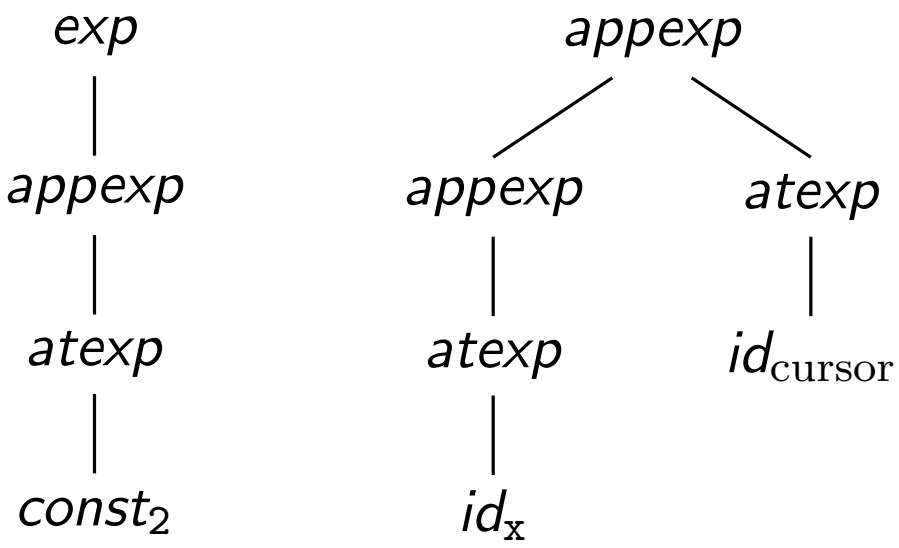
- カーソル位置を EOF とする。
- LALR(1) 構文解析をカーソル位置まで行う。
- EOF を初めて先読みした時に ␣ というトークンを EOF の前に挿入
- 2 回目に EOF を先読みした時点で、現在の還元の履歴とスタックの状態を返して構文解析を終了する。

構文解析の結果の例

スタックの情報

$$\texttt{let}, \texttt{val},\ id_{\texttt{x}}, =,\ exp,\ \texttt{in},\ appexp$$

還元の履歴



構文木の補完

構文解析の結果からトークンを補いながら完全な構文木を作成する。

構文規則に各非終端記号にダミー要素を持てるように変更する。

ダミー要素を追加した構文規則

$$
\begin{array}{lcl}
\mathit{start} & ::= & \mathit{exp} \\
\mathit{exp} & ::= & \mathit{appexp} \\
 & | & \texttt{fn}\ \mathit{id}\ \Rightarrow \mathit{exp} \\
 & | & [\,] \\
\mathit{appexp} & ::= & \mathit{atexp} \\
 & | & \mathit{appexp}\ \mathit{atexp} \\
\mathit{atexp} & ::= & \mathit{id} \\
 & | & \mathit{const} \\
 & | & (\mathit{exp}) \\
 & | & \texttt{let val}\ \mathit{id} = \mathit{exp}\ \texttt{in}\ \mathit{exp}\ \texttt{end} \\
 & | & [\,]
\end{array}
$$

(1) $s_1 = start$ なら終了。s_1 を右辺に含む各生成規則 $A \rightarrow \alpha$ において、$\alpha = \alpha_1 s_1 \alpha_2$ とする。$\alpha_1 = s_1 \ldots s_i$ となるものについて還元を行う。α_2 の中の非終端記号をダミー要素に変えた記号列をスタックに積み還元する。

(2) (1) に戻る。

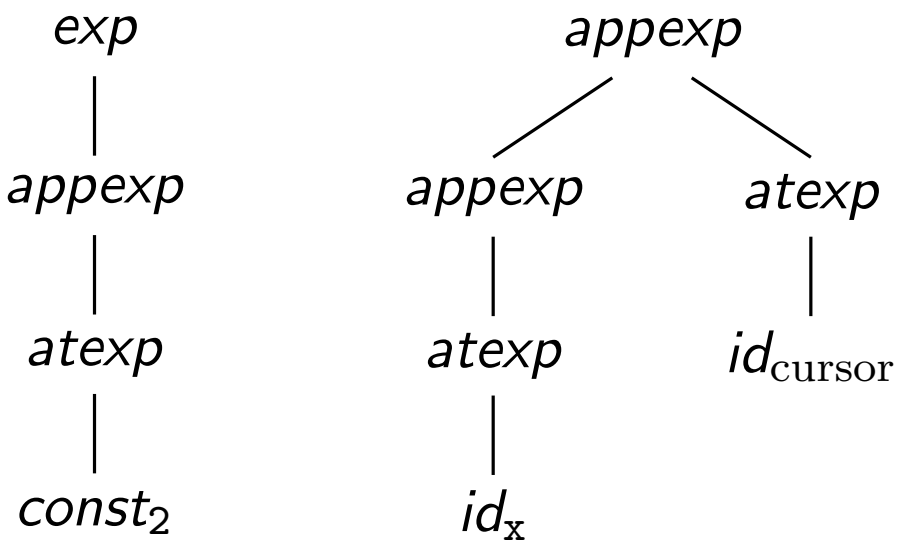
一般的に生成される構文木は無限になるので本研究ではしきい値を用いて有限個の構文木を生成することとした。

構文木補完の例

スタックの情報

$$\texttt{let}, \texttt{val},\ id_{\texttt{x}}, =,\ exp,\ \texttt{in},\ appexp$$

還元の履歴

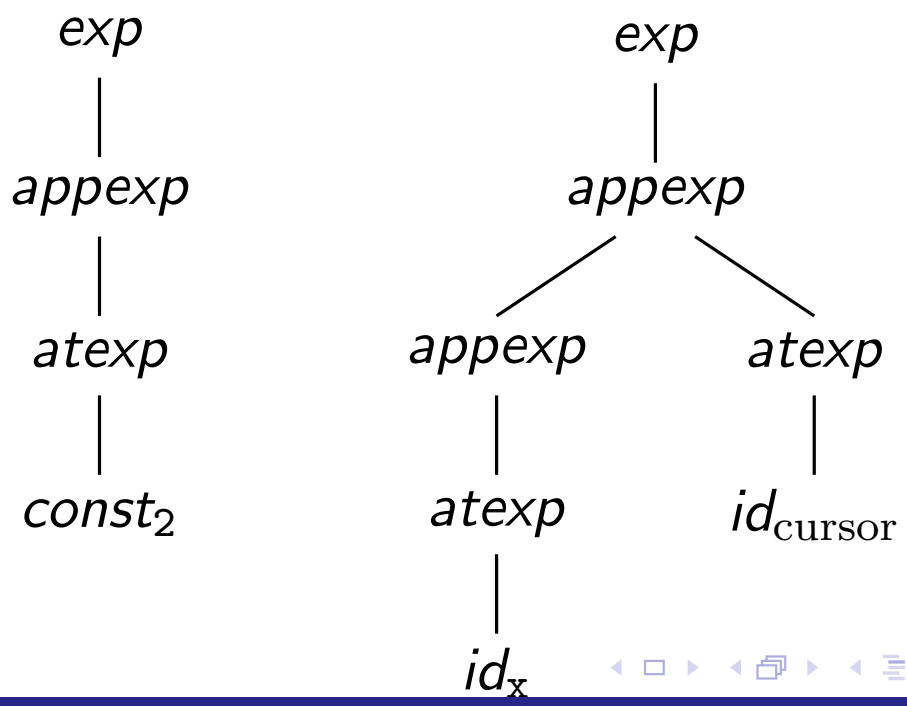


構文木補完の例:規則 2 を適用した場合

スタック

$$\texttt{let}, \texttt{val},\ id_{\texttt{x}}, =,\ exp,\ \texttt{in},\ exp$$

還元の履歴

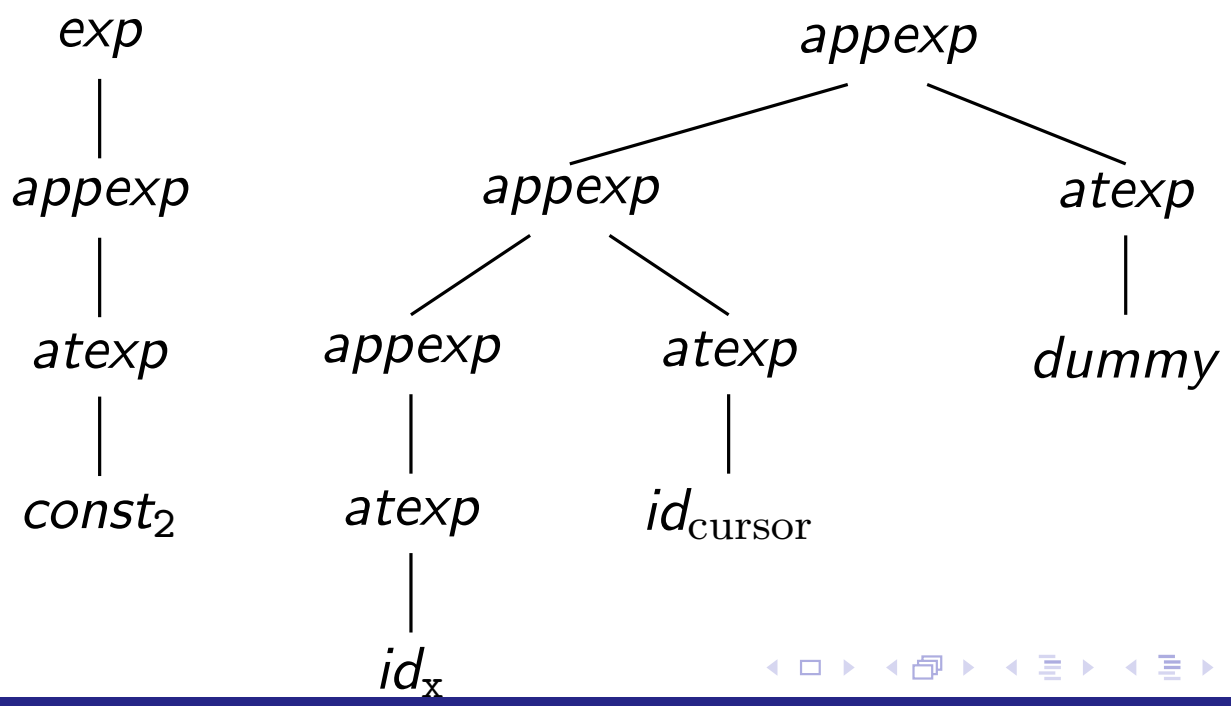


構文木補完の例:規則 5 を適用した場合

スタック

$$\texttt{let}, \texttt{val},\ id_{\texttt{x}}, =,\ exp,\ \texttt{in},\ appexp$$

還元の履歴



- let 式による変数宣言を複数記述できるように拡張

ネストを深くせずにプログラムが記述できるようになったため、
ある程度大きめのプログラムが記述できるように。

型システムの定義

$$\text{(const)} \quad \Gamma \triangleright c : \tau \quad (c : \tau \in \mathit{Const})$$

$$\text{(var)} \quad \Gamma\{x : \sigma\} \triangleright x : \tau \quad \text{if } \tau < \sigma$$

$$\text{(app)} \quad \frac{\Gamma \triangleright e_1 : \tau_1 \to \tau_2 \quad \Gamma \triangleright e_2 : \tau_1}{\Gamma \triangleright e_1\ e_2 : \tau_2}$$

$$\text{(abs)} \quad \frac{\Gamma\{x : \tau_1\} \triangleright e : \tau_2}{\Gamma \triangleright \lambda x.e : \tau_1 \to \tau_2}$$

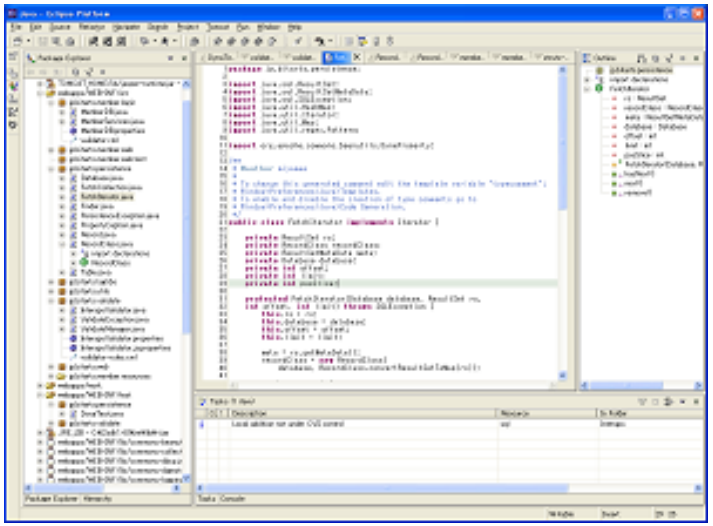
$$\text{(let)} \quad \frac{\Gamma \triangleright e_1 : \tau_1 \quad \Gamma\{x_1 : \mathit{Cls}(\Gamma, \tau_1)\} \triangleright e_2 : \tau_2}{\Gamma \triangleright \textbf{let}\ x_1 = e_1 \textbf{in}\ e_2 : \tau_2}$$

型推論後のプログラム

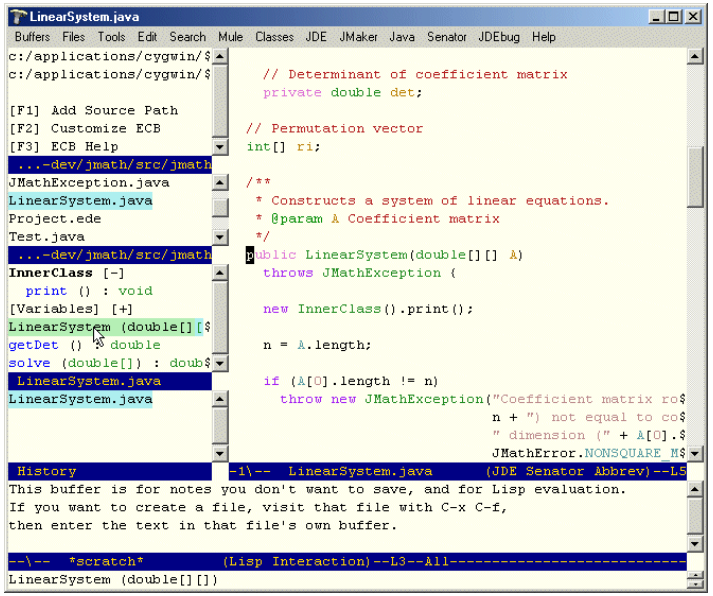
$$
\begin{array}{l}
\mathit{let}\ \mathit{val}\ \underline{xy}_{\mathit{int}\to\mathit{int}} = \mathit{fn}\ z \Rightarrow +\ z\ 1 \\
\quad \mathit{var}\ \underline{xx}_{\mathit{int}} = \underline{1}_{\mathit{int}} \\
\mathit{in}\ xy\ \llcorner\ [\,]\mathit{end}
\end{array}
$$

xy ␣ の型が int であり、関数型でないので
そのあとに引数をとることは不可能 ⇒ 型エラー

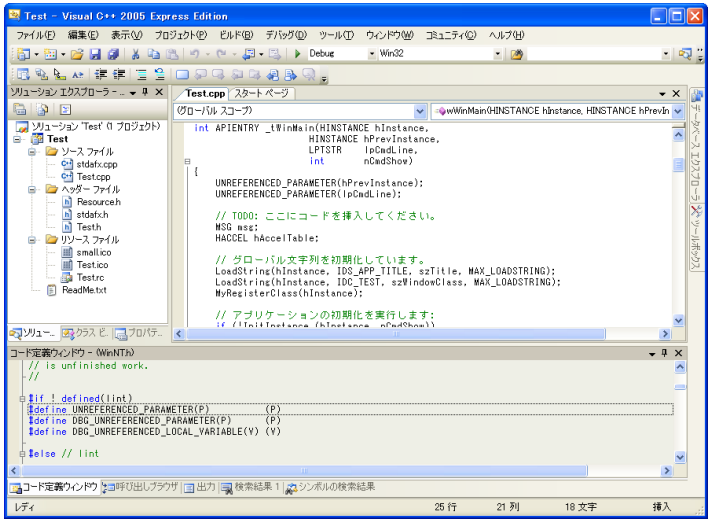
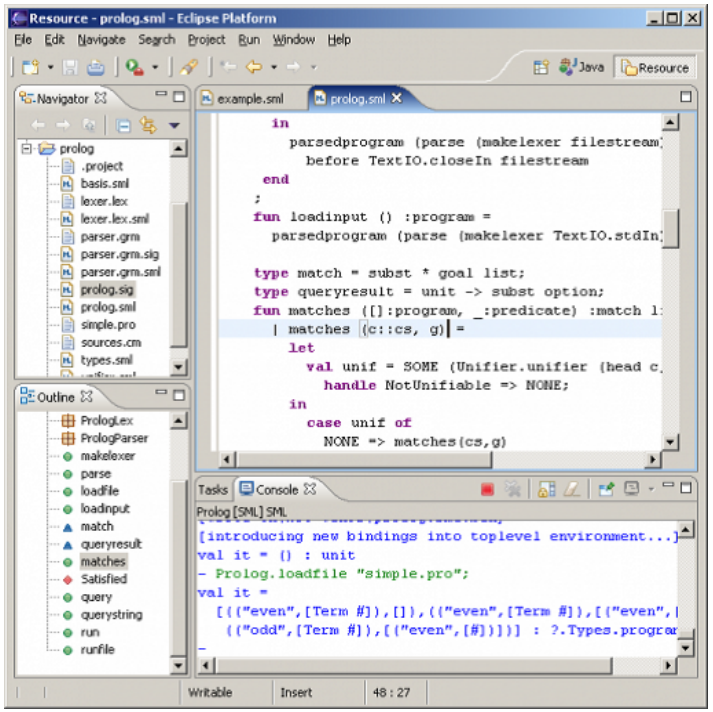
Java の開発環境:Eclipse



Java の開発環境:JDEE



C++の開発環境:Microsoft Visual C++



Standard ML の開発環境:ML-Dev